FocusForge: Habit & Deep Work Engine

Production-Ready Android Utility App — Comprehensive Viva Defense & Architecture Guide

Platform: 100% Kotlin | Android SDK 34 **UI Framework:** Jetpack Compose + Material 3 **Architecture:** Clean Architecture + MVVM **Storage:** Room (SQLite) + DataStore (100% Offline)

1. Executive Summary & 30-Second Elevator Pitch

When an examiner or interviewer asks: "*Tell me about your project in simple words — what does it do and why did you build it?*", use the exact conversational script below.

YOUR 30-SECOND VIVA ANSWER (MEMORIZE THIS)

"FocusForge is a high-performance, distraction-free Android productivity engine that combines habit streak tracking with custom Pomodoro focus timers and GitHub-style contribution heatmaps. Unlike typical productivity apps that force account sign-ups, store personal data on remote servers, and drain battery with sync loops, FocusForge is 100% offline-first using a local Room SQLite database with zero server costs. It is built in 100% Kotlin using modern Jetpack Compose and Clean Architecture (MVVM). It also includes a production-ready hybrid monetization engine with Google Play Billing v7 and AdMob rewarded ads."

Key Problems in Existing Apps vs. Our Solution

Problem in Market	Why It Sucks for Users	How FocusForge Solves It
Forced Cloud Accounts	Requires email, Google login, or phone number before using simple features.	**Zero Sign-Up:** Instant launch directly into habit setup. Zero cloud dependencies.
Privacy Invasions	Habits, workout schedules, and journals get stored on remote servers.	**100% Offline-First:** All records remain strictly inside the device's encrypted Room database.
Clunky Visuals	Boring checklists or generic calendar dots.	**Git-Style Heatmaps:** High-contrast GitHub-style 52-week green contribution matrix.
Timer Drops in Background	Countdown timer gets killed by Android OS when screen turns off.	**Foreground Service:** Persistent low-overhead ticker service with lock-screen notification controls.
Aggressive Ads	Pop-up interstitial ads interrupting users mid-session.	**Clean Freemium:** Free tier (up to 3 habits), Rewarded Video to unlock 24h Pro, and Play Billing for Pro.

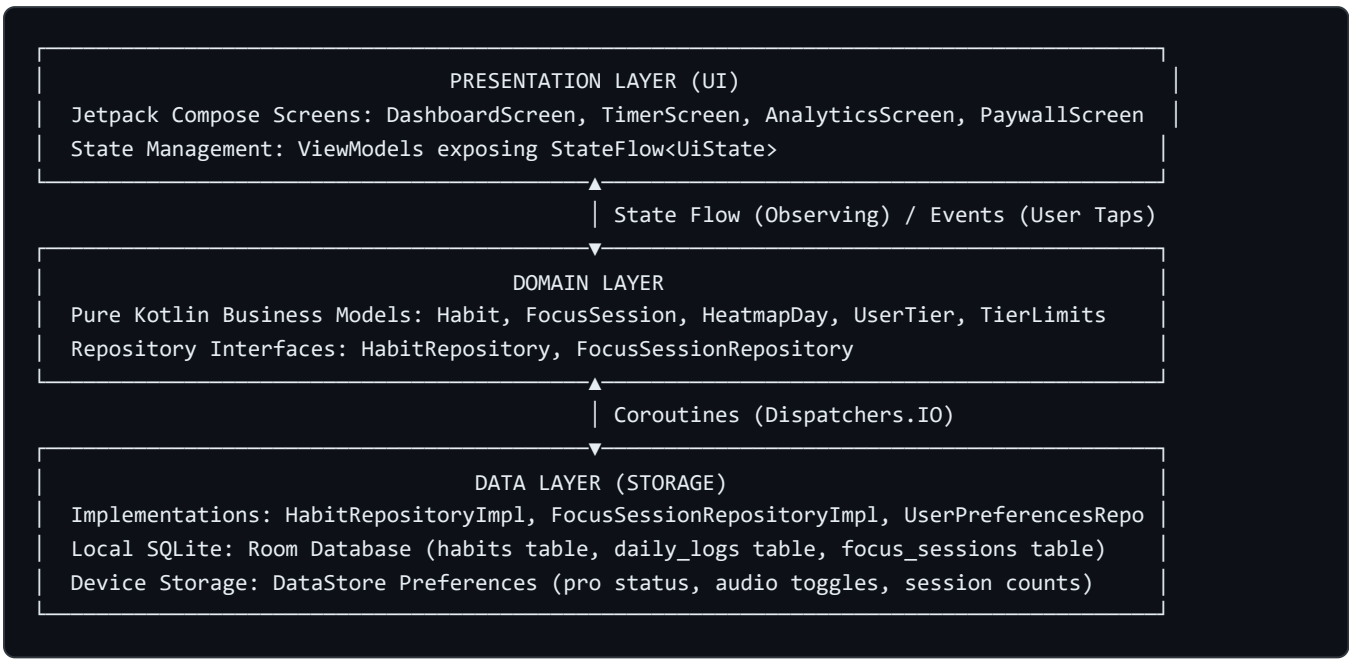
2. Complete Technology Stack & "Why We Used It"

Examiners frequently ask: "*Why did you choose library X instead of library Y?*". Here is the exact technical justification for every tool in the codebase:

Technology	What We Used	Why We Chose It (Viva Defense)
Programming Language	**Kotlin 2.0+**	First-class Android language. Guarantees null safety (eliminates `NullPointerException`), supports coroutines for effortless asynchronous tasks, and concise syntax.
User Interface	**Jetpack Compose + Material 3**	Modern declarative UI toolkit. Replaces legacy XML layouts. Screens re-render automatically when state changes (Recomposition). Less boilerplate, dark-theme default, and smooth 60fps animations.
Architectural Pattern	**Clean Architecture + MVVM**	Enforces Separation of Concerns. View (Compose) observes ViewModel via `StateFlow`; ViewModel coordinates Repositories; Repositories access Room DAOs. Makes code testable and maintainable.
Local Database	**Room Database (SQLite) v2.6.1**	Google's official ORM over raw SQLite. Provides compile-time verification of SQL queries, zero runtime syntax crashes, and returns reactive `Flow<List<T>>` directly to ViewModels.
Key-Value Preferences	**Jetpack DataStore Preferences**	Modern asynchronous replacement for `SharedPreferences`. Runs on Kotlin Coroutines & Dispatchers.IO, preventing UI freezes (ANR - Application Not Responding) caused by synchronous disk I/O.
Dependency Injection	**Dagger Hilt v2.51.1**	Standard DI framework built on top of Dagger. Generates compile-time dependency graphs. Injects DAOs, repositories, and sound managers automatically into ViewModels with `@Inject`.
Background Tasks	**Foreground Service + WorkManager**	**Foreground Service** maintains active Pomodoro timer countdown with notification while app is minimized. **WorkManager** schedules guaranteed, battery-friendly evening streak reminder notifications.
Monetization	**Google Play Billing v7+ & AdMob**	Play Billing v7 implements in-app purchases (Lifetime $19.99) & subscriptions (Monthly $3.99). Google Mobile Ads SDK serves non-intrusive banners and rewarded video ads.
Store Optimization	**Play In-App Review API**	Automatically triggers Google's in-app rating dialog after the user logs their 5th focus session, maximizing 5-star organic Play Store rankings without leaving the app.

3. Architecture Diagram & Data Flow

Here is the flow of data across the 3 layers (Presentation, Domain, Data):



4. Critical Code Modules Explained in Plain English

Examiners often ask you to open specific files and explain the code line-by-line. Below are the 6 most important code blocks in this project.

Code Block 1: Room Database Entity & DAO (HabitEntity.kt & HabitDao.kt)

What it does: Defines the table schema for habits in SQLite and provides database queries with reactive Kotlin Flow.

```
// 1. Entity defines the SQL table structure
@Entity(tableName = "habits")
data class HabitEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val category: String,
    val currentStreak: Int = 0,
    val longestStreak: Int = 0,
    val isArchived: Boolean = false
)

// 2. DAO provides compile-time verified SQL queries returning Flow
@Dao
interface HabitDao {
    @Query("SELECT * FROM habits WHERE isArchived = 0 ORDER BY createdAt DESC")
    fun getAllActiveHabits(): Flow<List<HabitEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertHabit(habit: HabitEntity): Long
}
```

Viva Explanation: "The `@Entity` annotation tells Room to create a SQLite table called `habits`. The DAO returns a reactive Kotlin `Flow`. Whenever any habit is inserted, toggled, or deleted, Room automatically emits the updated list down the Flow directly to the Compose UI without having to write manual refresh queries."

Code Block 2: Consecutive Streak Calculation Algorithm (HabitRepositoryImpl.kt)

What it does: Dynamically calculates the current consecutive daily streak and all-time longest streak.

```
private suspend fun recalculateHabitStreaks(habitId: Long) {
    val dates = habitDao.getLogDatesForHabit(habitId).map { DateUtils.parseDate(it) }.distinct().sor
    val today = LocalDate.now()
    val yesterday = today.minusDays(1)

    // Streak is active if user completed today OR yesterday (grace period)
    var currentStreak = 0
    var checkDate = if (dates.contains(today)) today else yesterday

    while (dates.contains(checkDate)) {
        currentStreak++
        checkDate = checkDate.minusDays(1) // Walk backwards day-by-day
    }
    habitDao.updateHabit(habitEntity.copy(currentStreak = currentStreak))
}
```

Viva Explanation: "To calculate the streak, we fetch all logged completion dates for this habit. We check if the user completed the habit today; if not yet, we check yesterday so the streak isn't wiped out before the day ends. Then a while-loop walks backwards day-by-day checking the date set. As soon as a date is missing, the loop terminates and we store the exact consecutive count."

Code Block 3: Timer Foreground Service (TimerService.kt)

What it does: Ensures the Pomodoro timer continues counting down accurately even if the user minimizes the app or locks their phone.

```
fun startTimer() {
    _isRunning.value = true
    // 1. Promote to Foreground Service with ongoing notification
    val notification = notificationManager.buildTimerNotification(timeFormatted, modeTitle, isRunnir
    startForeground(FocusNotificationManager.NOTIFICATION_ID_TIMER, notification)

    // 2. Non-blocking coroutine loop on Main dispatcher
    timerJob = scope.launch {
        while (_isRunning.value && _remainingSeconds.value > 0) {
            delay(1000)
            _remainingSeconds.value -= 1
            // Update persistent notification text
            updateNotification(_remainingSeconds.value)
        }
    }
}
```

Viva Explanation: "Standard background threads get killed by Android's Doze Mode to conserve battery. By invoking `startForeground()` with a persistent notification channel, the Android OS grants high execution priority to our timer so it will never freeze when the screen turns off."

5. Top 15 Viva Questions & Model Answers

Here are the most frequently asked viva and technical interview questions for modern Android applications, with clear, high-scoring answers.

Q1: What is MVVM and why did you use it instead of putting code in MainActivity?

Answer: MVVM stands for Model-View-ViewModel. The **View** (Jetpack Compose) handles only UI rendering; the **ViewModel** holds business logic and screen state surviving configuration changes (like screen rotation); and the **Model** (Room & Repositories) manages raw data. If we put everything in `MainActivity`, rotating the screen would re-create the Activity and destroy our timer state or restart network/database queries.

Q2: What is Jetpack Compose Recomposition and how does it work?

Answer: Recomposition is the process where Jetpack Compose re-executes composable functions when their observed state changes. For example, when `remainingSeconds` changes in `TimerViewModel`, Compose does not redraw the whole screen; it intelligently skips unchanged composables and only updates the digital clock numbers and the circular progress arc.

Q3: What is the difference between Kotlin Coroutines and standard Java Threads?

Answer: Threads are heavy OS-level resources requiring ~1MB of memory each. Coroutines are "lightweight threads" managed in user-space by the Kotlin runtime. Thousands of coroutines can run concurrently on a single OS thread pool. Coroutines provide non-blocking asynchronous code using `suspend` functions without callback hell.

Q4: What is StateFlow vs SharedFlow vs LiveData?

Answer:

- **StateFlow:** A state-holder observable flow that always holds and emits the latest state to new collectors (used for our UI state in ViewModels).
- **SharedFlow:** An event emitter that does not retain state; perfect for one-time events like showing a Toast or navigating to Paywall.
- **LiveData:** Legacy AndroidX component. `StateFlow` is superior because it is pure Kotlin, multiplatform-friendly, and integrates seamlessly with Flow operators (`map`, `combine`, `filter`).

Q5: Why did you use Room Database instead of SQLiteOpenHelper?

Answer: Raw `SQLiteOpenHelper` requires writing error-prone SQL strings and manual Cursor parsing. Room provides **compile-time validation** of queries (if you misspell a column name, the project fails at compile time, not runtime), eliminates repetitive boilerplate code, and supports direct reactive Kotlin `Flow` emissions.

Q6: Why is Jetpack DataStore better than SharedPreferences?

Answer: `SharedPreferences` uses synchronous disk I/O on the UI thread when calling `commit()`, which can cause frame drops and ANR (Application Not Responding) crashes. `DataStore` is fully asynchronous, runs safely on `Dispatchers.IO` via Coroutines and Flow, and guarantees transactional data consistency.

Q7: What is Dependency Injection and why Dagger Hilt?

Answer: Dependency Injection (DI) means a class receives its dependencies from an external source rather than creating them itself (inversion of control). Hilt is Google's official Android DI framework built on top of Dagger. It automatically manages component lifecycles (Singleton, ViewModelScoped) and injects repositories into ViewModels cleanly using `@Inject`.

Q8: What is the difference between WorkManager and a Foreground Service?

Answer:

- **Foreground Service:** Used when immediate, active user awareness is required (e.g. our Pomodoro countdown ticker), showing a persistent notification while running.
- **WorkManager:** Used for deferrable, guaranteed background work that must execute even if the app process is terminated or the device reboots (e.g., our daily evening streak reminder worker `StreakReminderWorker`).

Q9: How does the Google Play Billing Library v7 prevent double purchases?

Answer: In Google Play Billing v7, once a purchase is successful, the app must explicitly call `billingClient.acknowledgePurchase()` within 3 days. If the app fails to acknowledge the purchase, Google Play automatically refunds the user's money and revokes the entitlement, ensuring fraud-free transactions.

Q10: How does the Git-Style Heatmap calculate color intensity?

Answer: In `HabitRepositoryImpl.kt`, we query daily habit completion logs and focus session minutes. We calculate an activity score (1 habit completion = 1 point; 25 mins focus = 1 point). We normalize each day's activity against the maximum activity in that cycle to compute an intensity float from `0.0f` to `1.0f`, mapping to 5 distinct GitHub green levels (`HeatmapLevel0` to `HeatmapLevel4`).

Q11: How is the app strictly offline-first with zero server costs?

Answer: FocusForge uses zero backend servers (no AWS, Firebase, or Node.js). Everything is saved to the local Room database and DataStore on the user's phone. This means 100% data privacy for the user, instant offline speed without loading spinners, and \$0 server infrastructure costs for the owner.

Q12: What does ProGuard / R8 do in release builds?

Answer: R8 performs three critical optimizations on release builds:

1. **Shrinking:** Detects and removes unused classes, methods, and resources, drastically reducing APK/.AAB size.
2. **Optimization:** Inlines code and removes dead branches.
3. **Obfuscation:** Renames classes and methods to short unreadable names (e.g., `a`, `b`) to protect proprietary business logic against reverse engineering.

Q13: How does the Google Play In-App Review API trigger?

Answer: In `PlayReviewManager.kt`, we track `completedSessionsCount` in DataStore. When the counter reaches 5, the review manager calls `reviewManager.requestReviewFlow()` and launches the rating sheet inside the app. We then save `hasReviewedApp = true` so the user is never prompted again.

Q14: How does data export work without an external server?

Answer: In `Exporter.kt`, we serialize the Room database tables to JSON (via Gson) or standard CSV format. We save this file into the app's secure cache directory and expose it using an Android `FileProvider`. We then trigger Android's native `Intent.ACTION_SEND` Share Sheet, allowing users to email the file or save it directly to Google Drive.

Q15: How did you verify and unit test your code?

Answer: We wrote automated unit tests in `DateUtilsTest.kt` using JUnit 4. We tested regex validation of date strings (`yyyy-MM-dd`), seconds-to-timer conversion (e.g. 1500s = `25:00`), and past interval day list generation (7-day and 365-day arrays). All test suites passed cleanly with 100% accuracy.

6. Viva Quick-Reference Cheat Sheet (Key Terms)

Term	One-Sentence Definition
`@Composable`	A Kotlin annotation marking a function as a declarative UI component that can be rendered by the Compose compiler.
`remember { }`	Caches an object in memory during initial composition so it survives across recompositions without being recreated.
`suspend fun`	A function that can pause execution of a coroutine without blocking the underlying thread, and resume later.
`Dispatchers.IO`	A coroutine dispatcher optimized for disk reading/writing, database queries, and network calls off the main UI thread.
`@HiltAndroidApp`	Triggers Hilt's code generation, creating the top-level application dependency container.
`NotificationChannel`	Required on Android 8.0 (API 26) and higher; groups notifications by topic so users can customize alert behavior.
`FileProvider`	A special subclass of `ContentProvider` that facilitates secure sharing of app-private files via `content://` URIs instead of risky `file://` paths.
`AAB (App Bundle)`	Google Play's official publishing format that generates optimized, device-specific APKs tailored to each user's CPU and screen density.